8.1. Диаграммы C4.

C1: Диаграмма контекстная.

Контекстная диаграмма отражает взаимодействие системы «НРИ-Помощник» с внешними пользователями и источниками данных.

Основные участники системы:

1. Игрок

Использует приложение для создания листа персонажа и передачи его геймммастеру.

2. Геймммастер

Использует систему для ведения сессии, управления персонажами, NPC и событиями боя.

3. Локальное хранилище

Файловая система или база SQLite, в которой хранятся данные персонажей, сессий и заметок.

Система работает полностью локально и не требует сетевого подключения.

Изображение.

C2: Диаграмма контейнерная.

Контейнерная диаграмма показывает архитектуру системы на уровне основных подсистем.

Контейнеры системы:

1. User Interface
Пользовательский интерфейс приложения.
Позволяет игрокам и геймастеру работать с персонажами, запускать боевые сцены и управлять сессией.

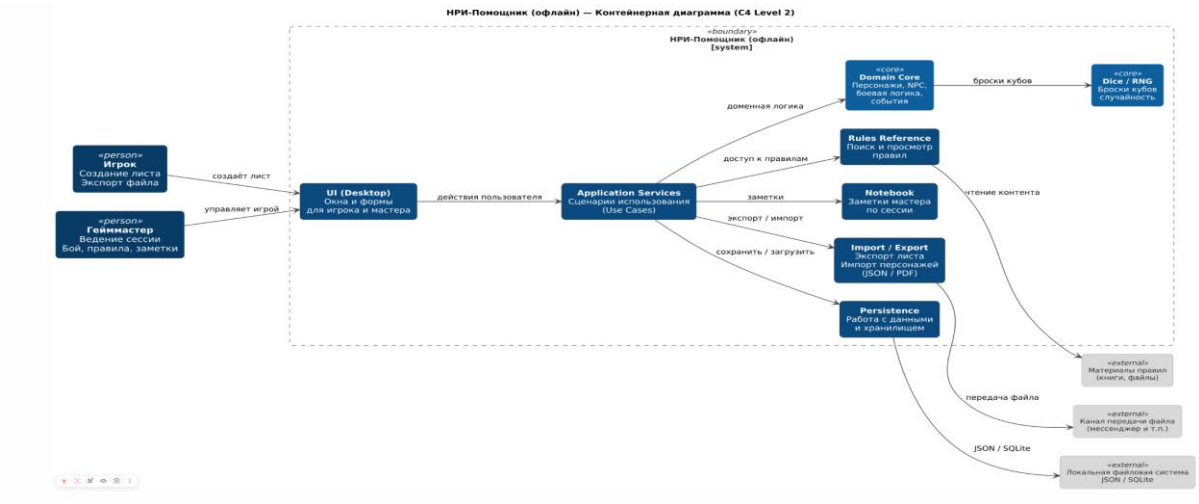
2. Application Services
Слой сценариев использования.
Содержит операции создания персонажей, запуска боя, изменения характеристик и работы с игровыми событиями.

3. Domain Core
Основная игровая логика системы.
Реализует механику персонажей, правила боя, броски кубов и вычисления характеристик.

4. Import Export
Контейнер обмена файлами персонажей.
Обеспечивает экспорт и импорт листов персонажей между игроками и геймастером.

5. Persistence
Слой хранения данных.
Использует JSON и SQLite для сохранения персонажей, сессий и журналов событий.

Изображение.



C3: Компонентная.

Диаграмма 3A — Component: Domain Core

Диаграмма раскрывает контейнер Domain Core на внутренние компоненты и показывает два параллельных пути изменения состояния персонажей и боя.

Изменения по механике происходят через Rules Engine, который выполняет вычисления характеристик и результаты бросков кубов.

Второй путь связан с ручным вмешательством геймастера. Для этого используется компонент Stat Override Service, который позволяет корректировать параметры персонажей в административном режиме. При этом система сохраняет контроль целостности данных и журналирует изменения.

Таким образом система поддерживает как автоматическую механику правил, так и ручное управление геймастера.

Изображение.

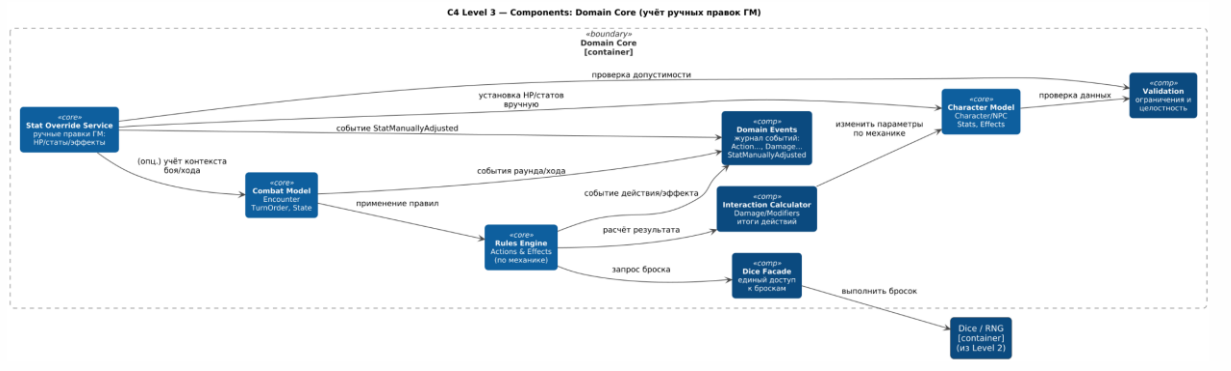


Диаграмма 3B — Component: Import/Export

Контейнер Import Export отвечает за обмен листами персонажей.

Последовательность обмена:

- Игрок экспортирует лист персонажа
- Файл передаётся геймастеру
- Геймастер импортирует файл в систему
- Происходит проверка структуры и версии формата
- Данные преобразуются в доменную модель
- Лист сохраняется в библиотеке персонажей

Контейнер обеспечивает контроль структуры данных и предотвращает повреждение системы некорректными файлами.

Изображение.

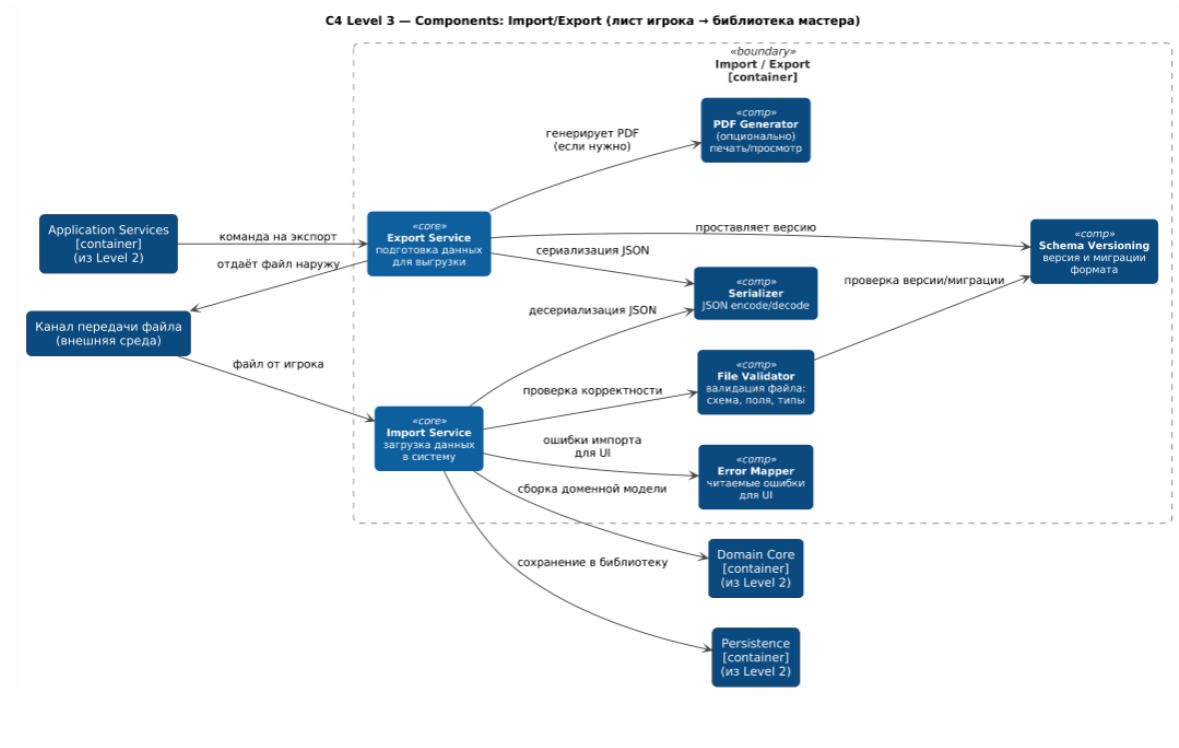


Диаграмма 3C — Component: Application Services

Контейнер Application Services представляет набор сценариев использования.

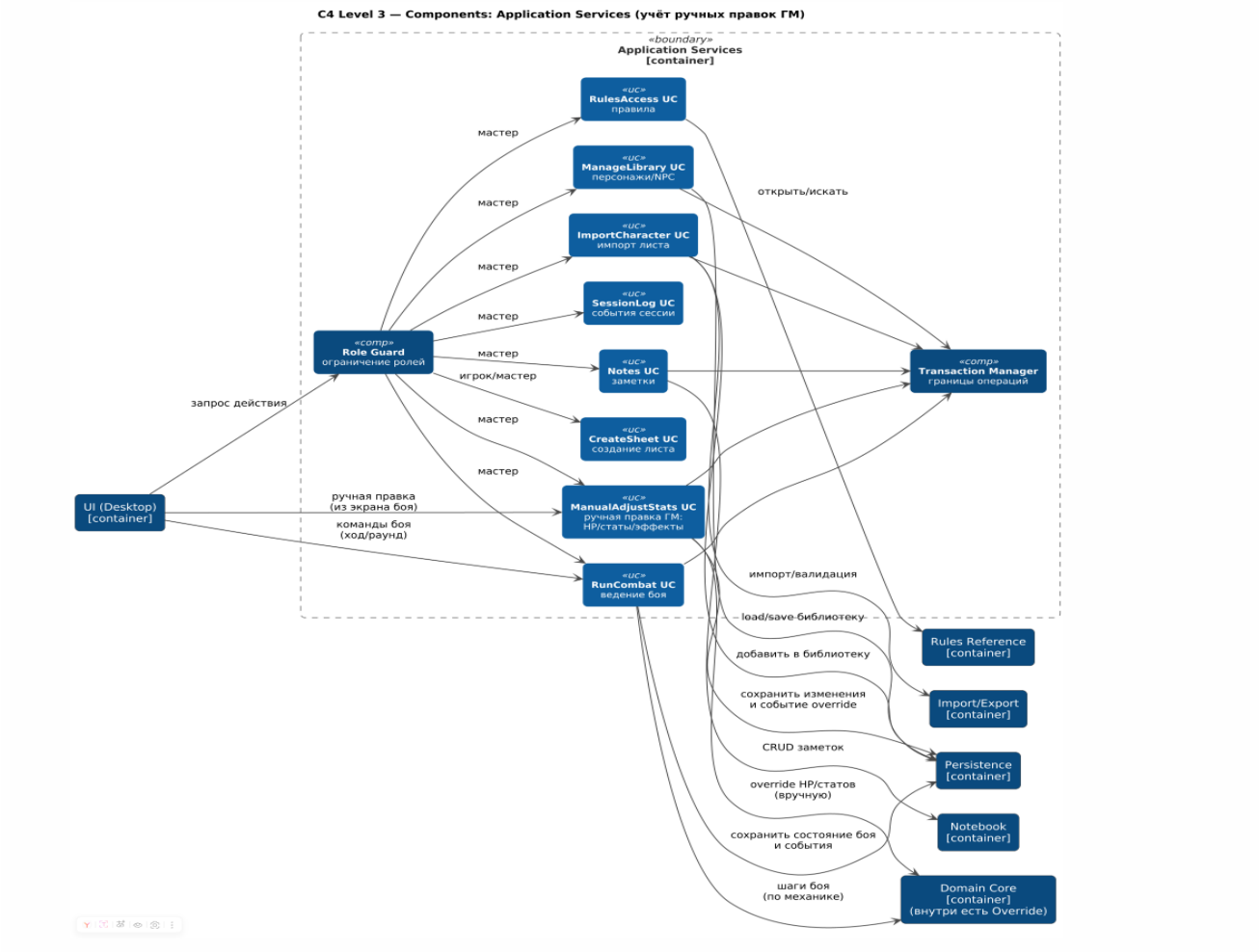
Каждый сценарий:

- ограничивается по ролям пользователя
- имеет транзакционные границы
- использует доменную логику
- сохраняет данные через слой хранения

Примеры сценариев:

- создание персонажа
- запуск боя
- изменение характеристик
- добавление NPC
- ведение журнала событий

Изображение.



C4: Диаграмма Кода.

Диаграмма 4A - Code: Domain Core

Диаграмма уровня кода отражает структуру классов доменной логики.

Основные сущности:

- Character
- NPC
- Combat Encounter
- Dice Roll
- Rule Engine
- Status Effect
- Stat Modifier

Эти классы реализуют механику персонажей и боевых взаимодействий.

Изображение.

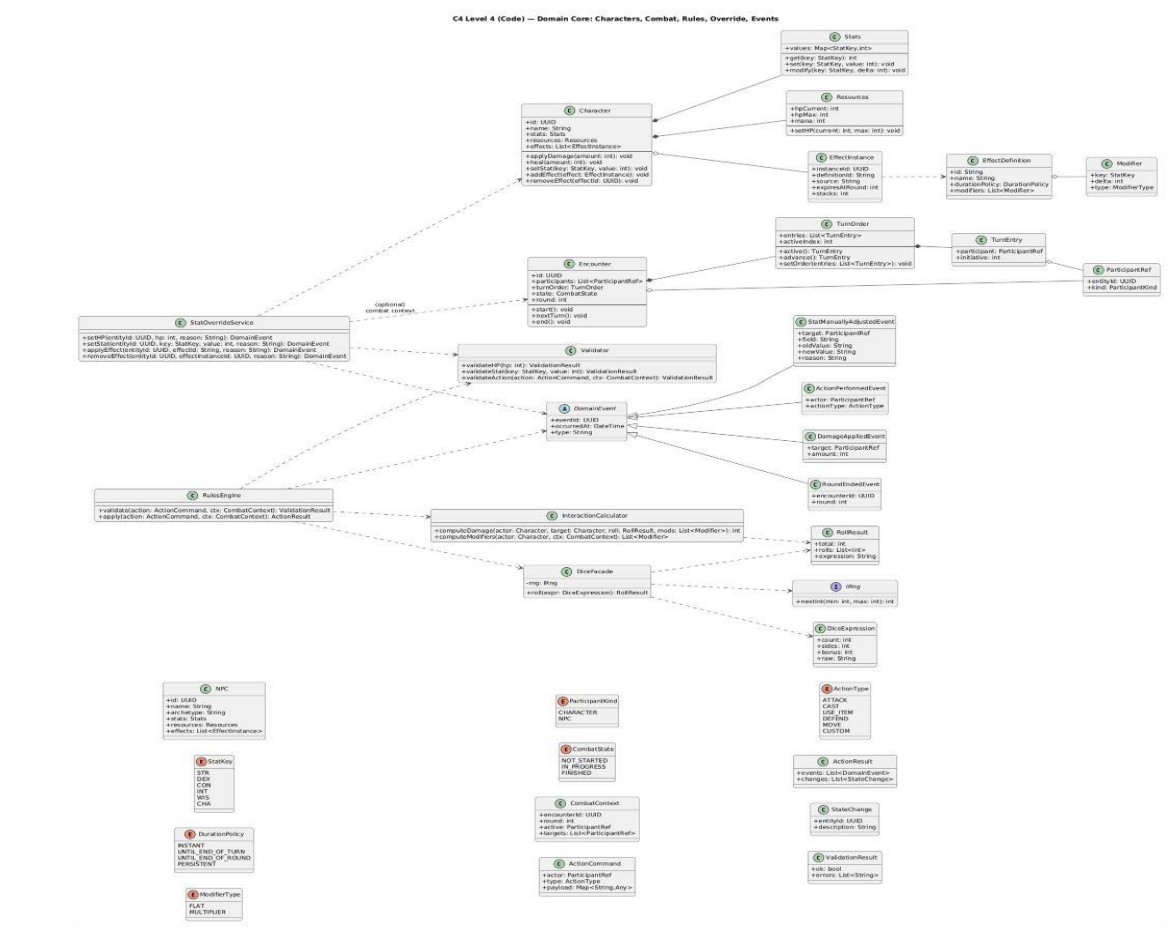


Диаграмма 4B – Code: Application Services

Слой Application Services реализует сценарии использования системы.

Основные классы:

- Character Service
- Combat Service
- Session Service
- Import Export Service

Каждый сервис принимает команды из пользовательского интерфейса, проверяет права пользователя и выполняет операции над доменной моделью.

Изображение.

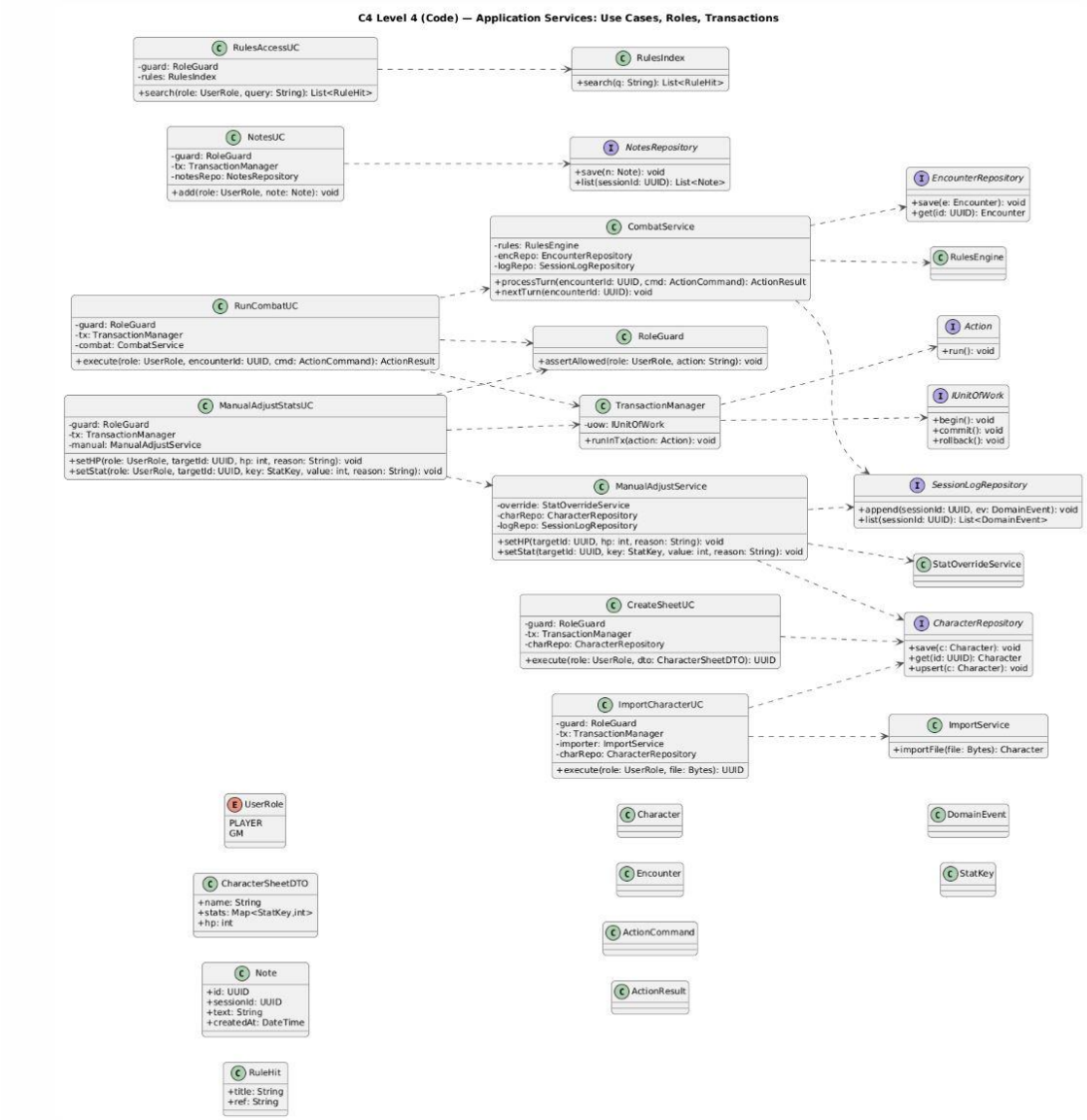


Диаграмма 4C- Code: Import/Export

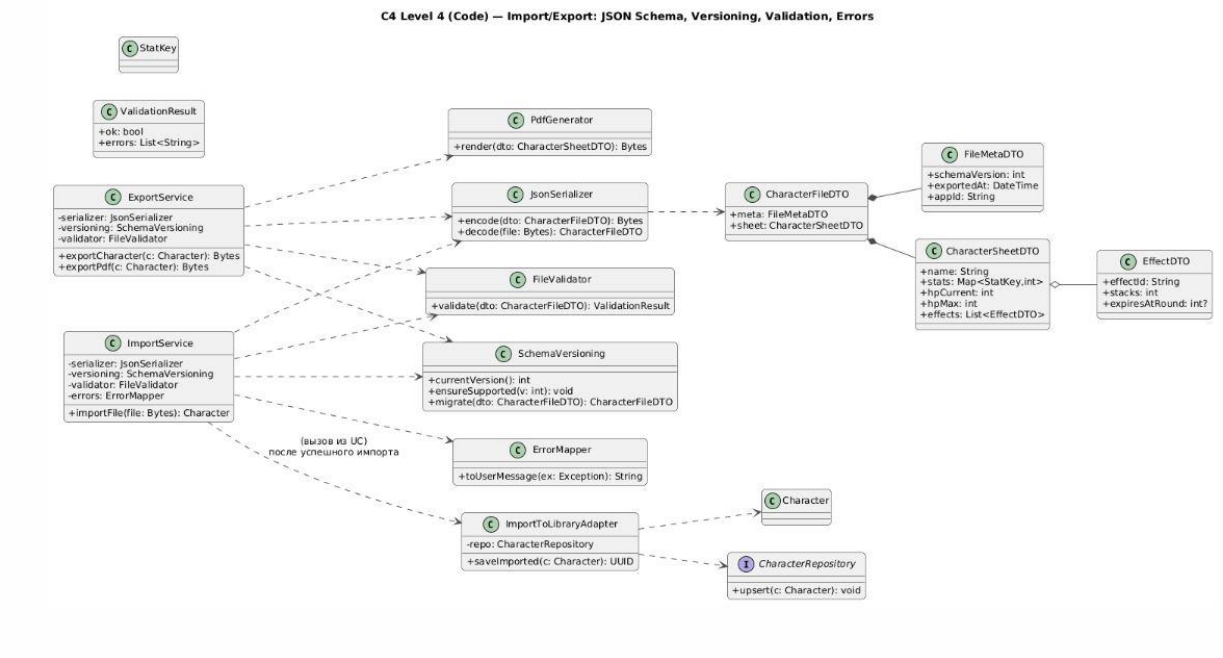
Контейнер обмена файлами реализует последовательный конвейер обработки данных.

Этапы обработки:

- подготовка данных к экспорту
- сериализация
- указание версии формата
- валидация структуры
- десериализация при импорте
- обработка ошибок
- сохранение в библиотеку персонажей

Это обеспечивает стабильность обмена файлами между участниками игры.

Изображение.



8.2. Диаграмма классов (эскиз главных классов системы)

Диаграмма классов отражает логическую структуру основных сущностей приложения «НРИ-Помощник». Она описывает ключевые объекты системы, их свойства, методы и взаимосвязи. Основная цель данной модели заключается в представлении структуры доменной области, связанной с управлением персонажами, игровыми сессиями и боевыми столкновениями.

Архитектура классов построена вокруг нескольких ключевых сущностей.

Основные доменные сущности

Класс Character

Класс Character представляет игрового персонажа. Он содержит основные характеристики персонажа, параметры здоровья, а также список активных эффектов и способностей.

Основные поля:

- id - уникальный идентификатор персонажа
- name - имя персонажа
- level - уровень персонажа
- healthPoints - текущее здоровье
- attributes - набор характеристик
- effects - список активных эффектов

Основные методы:

- takeDamage() - уменьшение здоровья персонажа
- heal() - восстановление здоровья
- applyEffect() - применение эффекта
- removeEffect() - удаление эффекта

Этот класс является центральным элементом всей доменной модели.

Класс NPC

Класс NPC представляет неигрового персонажа.

Он имеет структуру, аналогичную классу Character, но используется для описания противников или вспомогательных персонажей.

Основные поля:

- name
- healthPoints
- attributes
- abilities

NPC участвует в боевых столкновениях наравне с игровыми персонажами.

Класс CombatEncounter

Класс CombatEncounter описывает боевое столкновение.

Он содержит список участников боя, текущий ход, а также журнал событий.

Основные поля:

- id
- participants
- initiativeOrder
- combatLog

Основные методы:

- startCombat()
- nextTurn()
- endCombat()
- applyAttack()

Этот класс управляет всей логикой боя.

Класс DiceEngine

DiceEngine реализует механику бросков кубов.

Этот модуль используется при расчёте атак, проверок характеристик и других игровых действий.

Основные методы:

- rollDice()
- rollMultiple()
- calculateResult()

Класс RulesEngine

RulesEngine реализует игровую механику и вычисление результатов действий.

Он использует DiceEngine и характеристики персонажей.

Основные функции:

- расчёт успешности атаки
- проверка характеристик
- обработка эффектов
- модификация значений характеристик

Класс StatusEffect

Класс StatusEffect представляет временные эффекты, которые могут влиять на персонажа.

Примеры эффектов:

- яд
- оглушение
- усиление характеристик

Основные поля:

- name
- duration
- modifier

Класс Session

Session представляет игровую сессию.

Он содержит информацию о текущей игре и участниках.

Основные поля:

- sessionId
- characters
- npcList
- combatEncounters
- notes

Класс Notes

Notes представляет систему заметок геймастера.

Она используется для хранения информации о событиях, сюжете и игровых изменениях.

Связи между классами

Между объектами системы существуют следующие отношения.

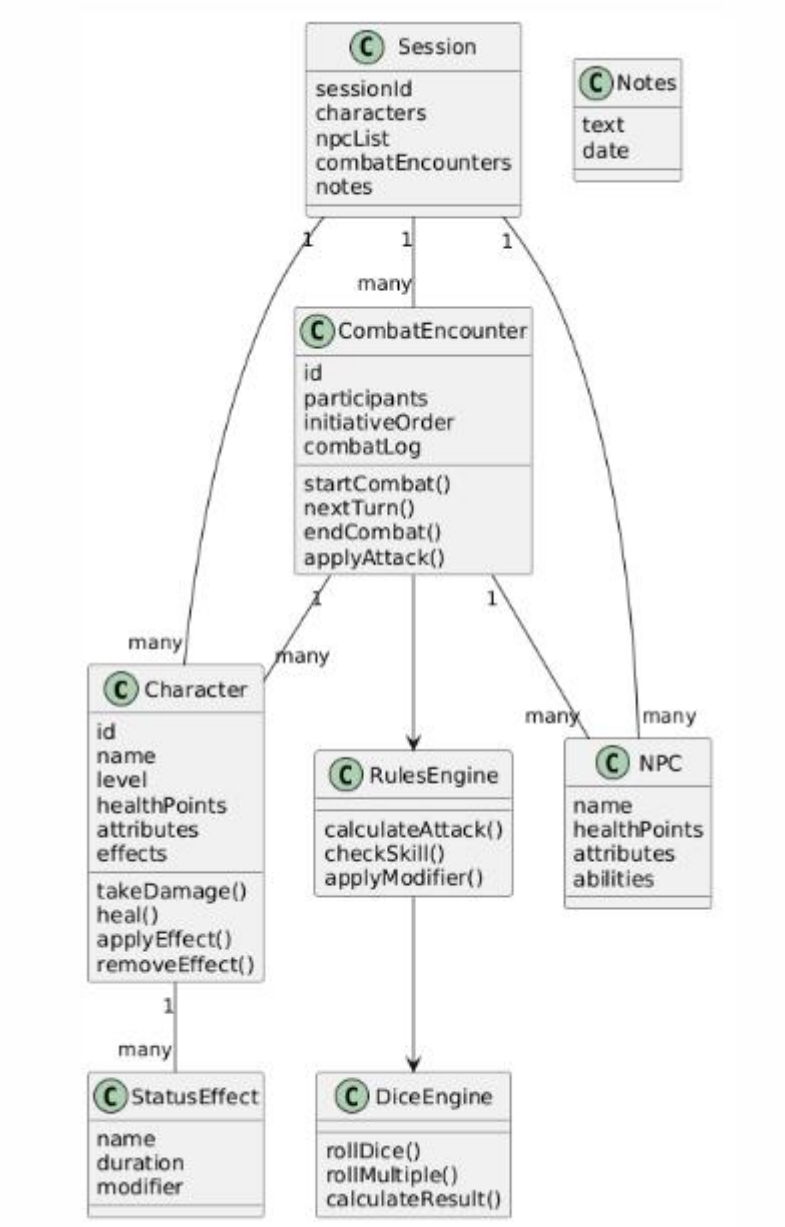
Character связан с StatusEffect через отношение один ко многим.

CombatEncounter связан с Character и NPC через список участников.

Session агрегирует все основные объекты системы.

RulesEngine использует DiceEngine для вычисления результатов.

Изображение.



8.3. Контейнеры системы

Контейнерное представление системы позволяет показать, из каких крупных функциональных частей состоит приложение «НРИ-Помощник (офлайн)», как именно между ними распределяется ответственность и каким образом организован обмен данными между пользовательским интерфейсом, прикладной логикой, доменной моделью и хранилищем. В рамках данного уровня архитектурного описания система рассматривается не на уровне отдельных классов, а на уровне крупных программных блоков, каждый из которых выполняет собственную роль в общей работе приложения.

Контейнерная модель особенно важна для данного проекта, поскольку приложение ориентировано на автономную локальную работу, поддерживает разные пользовательские роли, а также должно обеспечивать устойчивую обработку игровых данных, связанных с персонажами, NPC, игровыми сессиями, заметками, боевыми событиями и файлами обмена. Вследствие этого разбиение на контейнеры позволяет не только структурировать реализацию, но и показать преподавателю, что система проектировалась как набор согласованных подсистем, а не как единый неразделённый программный модуль.

Общая логика контейнерной архитектуры

В основе системы лежит разделение на несколько ключевых контейнеров:

- User Interface
- Application Services
- Domain Core
- Import/Export
- Persistence
- Reference/Rules Data
- Session Log

Каждый из этих контейнеров решает собственную группу задач. При этом контейнеры не дублируют ответственность друг друга, а взаимодействуют по определённым маршрутам. Пользователь работает через интерфейс, интерфейс передаёт команды в прикладной слой, прикладной слой инициирует сценарий использования, доменное ядро выполняет расчёты и логику игровых действий, после чего результат сохраняется в хранилище и, при необходимости, фиксируется в журнале событий.

Такой подход делает систему более понятной, удобной для сопровождения и расширения. Например, при изменении механики боя изменения вносятся прежде всего в Domain Core, а при изменении внешнего вида приложения

основная переработка затрагивает User Interface. В свою очередь, модуль импорта и экспорта может развиваться отдельно, не затрагивая непосредственно механизмы расчёта боя.

Контейнер User Interface

Контейнер User Interface представляет пользовательскую часть приложения. Он является точкой непосредственного взаимодействия с системой и предназначен для работы игрока и геймастера.

Через данный контейнер выполняются следующие действия:

- создание нового листа персонажа;
- просмотр библиотеки персонажей;
- редактирование характеристик;
- создание и запуск боевого столкновения;
- ведение сессии;
- просмотр журнала событий;
- импорт и экспорт листов персонажей;
- работа со справочными разделами и заметками.

С точки зрения архитектуры данный контейнер не должен самостоятельно реализовывать механику игры. Его задача заключается в получении действий пользователя, отображении текущего состояния данных и передаче команд в прикладной слой. Это важно, поскольку интерфейс может меняться без изменения внутренней логики работы программы.

Для проекта «НРИ-Помощник» контейнер User Interface особенно важен, так как система ориентирована не только на хранение данных, но и на оперативное использование во время игровой сессии. Следовательно, интерфейс должен быть достаточно простым, чтобы геймастер мог быстро переключаться между персонажами, столкновениями и заметками, не тратя лишнее время на навигацию.

Контейнер Application Services

Контейнер Application Services представляет слой прикладных сценариев использования. Именно здесь собираются воедино пользовательские действия, правила доступа, последовательность шагов и работа с другими контейнерами.

Если User Interface отвечает на вопрос, что пользователь нажал или открыл, то Application Services отвечает на вопрос, что именно нужно выполнить в рамках бизнес-сценария. Например, команда «создать персонажа» включает

не просто сохранение набора полей, а целый прикладной сценарий: принять данные из формы, проверить корректность структуры, при необходимости инициировать расчёт производных характеристик, передать объект в хранилище и вернуть результат в интерфейс.

В рамках данного контейнера реализуются такие сценарии, как:

- CreateCharacterUseCase;
- UpdateCharacterUseCase;
- ImportCharacterUseCase;
- ExportCharacterUseCase;
- StartCombatUseCase;
- ApplyActionUseCase;
- EndCombatUseCase;
- CreateSessionUseCase;
- SaveNoteUseCase;
- ApplyManualOverrideUseCase.

Здесь же может находиться проверка роли пользователя. Например, некоторые операции доступны только геймастеру: ручное изменение значений характеристик, управление NPC, фиксация событий боя, импорт листов в библиотеку кампании. Игрок, в свою очередь, ограничивается созданием собственного персонажа, просмотром своего листа и экспортом файла.

Таким образом, контейнер Application Services образует координационный слой системы. Он не хранит данные как таковые и не выполняет глубокие механические расчёты сам по себе, но управляет процессом выполнения операций и распределяет вызовы между доменной логикой, модулем импорта и экспорта и контейнером хранения.

Контейнер Domain Core

Контейнер Domain Core является центральной частью приложения. В нём реализована вся основная логика настольно-ролевой системы, связанная с персонажами, характеристиками, эффектами, столкновениями, бросками кубов и обработкой игровых правил.

Если рассматривать систему как программный инструмент помощи в проведении игры, то именно Domain Core определяет, каким образом приложение понимает саму игровую механику. Здесь располагаются сущности и сервисы, которые отвечают за:

- представление персонажа и NPC;

- хранение характеристик и производных параметров;
- вычисление результатов бросков;
- применение модификаторов;
- обработку статусов и эффектов;
- расчёт урона, лечения и изменений состояний;
- управление очередностью хода;
- ведение внутреннего состояния боевого столкновения.

Особое значение в рамках данного проекта имеет то, что Domain Core поддерживает два режима изменения состояния:

- изменение по игровой механике;
- ручная корректировка со стороны геймастера.

Первый режим реализуется через движок правил и расчётов. Второй необходим потому, что в реальной практике НРИ ведущий может вручную корректировать значения из-за особенностей сценария, домашних правил или игровой интерпретации ситуации. При этом система должна не просто допускать такое изменение, а делать это контролируемо, с журналированием и сохранением согласованности данных.

Именно поэтому Domain Core не является простой совокупностью классов. Это логический центр приложения, в котором сосредоточены игровые сущности и механизмы принятия изменений состояния.

Контейнер Import/Export

Контейнер Import/Export предназначен для контролируемого обмена листами персонажей и связанными данными между участниками игрового процесса.

Для офлайн-приложения этот контейнер имеет особое значение, поскольку передача листов между игроком и геймастером осуществляется не через онлайн-синхронизацию, а через файл. Следовательно, система должна обеспечивать безопасный и понятный процесс обмена.

Контейнер выполняет следующие задачи:

- формирование экспортируемого файла персонажа;
- сериализация данных в выбранный формат;
- добавление метаинформации о версии структуры;
- валидация входящего файла при импорте;
- десериализация данных;
- преобразование структуры файла в доменную модель;
- выдача понятных ошибок при несовместимости формата;

- передача корректных данных в слой сохранения.

Практический смысл этого контейнера состоит в том, что пользователь не работает напрямую с базой или внутренней моделью программы. Он взаимодействует с файлом, который должен быть достаточно простым для передачи, но при этом достаточно строгим с точки зрения структуры, чтобы система могла корректно его разобрать.

Таким образом, Import/Export выполняет роль контролируемого шлюза между внешним файлом и внутренней моделью приложения.

Контейнер Persistence

Контейнер Persistence отвечает за долговременное хранение данных приложения. В проекте он реализует работу с локальной файловой системой и базой SQLite, а также может использовать JSON для обменных или промежуточных структур.

Через данный контейнер сохраняются:

- листы персонажей;
- данные NPC;
- игровые сессии;
- боевые столкновения;
- пользовательские заметки;
- журнал изменений;
- справочные данные;
- настройки приложения.

Основная задача контейнера заключается в том, чтобы скрыть технические детали хранения от остальных слоёв системы. Это означает, что Application Services и Domain Core не должны зависеть от конкретного способа записи данных. Они работают с логическими операциями сохранения и чтения, а техническая реализация этих операций инкапсулируется внутри Persistence.

Такой подход полезен и с точки зрения расширяемости. Даже если в будущем формат хранения будет изменён, например часть данных будет перенесена из файлов JSON в таблицы SQLite или наоборот, остальная архитектура сохранит устойчивость и не потребует полной переработки.

Контейнер Reference/Rules Data

Для работы приложения необходим набор справочных и нормативных данных. В эту группу входят игровые правила, таблицы модификаторов, шаблоны расчётов, типы эффектов, варианты бросков и другие

структурированные сведения, которые не являются данными одной конкретной сессии, но используются доменной логикой постоянно.

Данный контейнер или логически выделенный блок хранения обеспечивает:

- доступ к справочнику игровых правил;
- чтение предустановленных типов эффектов;
- использование таблиц модификаторов;
- хранение конфигурации формул и вычислительных правил;
- поддержку расширения игровых наборов.

Его выделение позволяет не смешивать живые игровые данные со справочной основой системы. Вследствие этого логика расчётов становится более прозрачной, а обновление справочной части не затрагивает напрямую записи о персонажах и сессиях.

Контейнер Session Log

Журнал сессии и боя является важной частью приложения, особенно в условиях работы геймастера. Этот контейнер можно рассматривать как отдельный блок или подсистему фиксации событий, связанных с изменением состояния игровых объектов.

Он хранит:

- ходы участников;
- результаты бросков;
- применение эффектов;
- изменение характеристик;
- ручные правки геймастера;
- события начала и завершения столкновения.

Наличие такого контейнера повышает управляемость системы. Геймастер может вернуться к последовательности действий, проверить, в какой момент было изменено состояние персонажа, и восстановить логику происходивших событий. Для проектного практикума это также важно как показатель того, что система проектировалась не только как калькулятор характеристик, но и как инструмент поддержки процесса игры.

Взаимодействие контейнеров

Работа контейнеров может быть представлена как последовательность типового сценария.

Сначала пользователь через User Interface инициирует действие. Например, геймастер создаёт боевое столкновение или игрок формирует лист персонажа. Далее команда передаётся в Application Services, где определяется конкретный сценарий использования. После этого прикладной слой обращается к Domain Core для выполнения игровых расчётов и изменения состояния сущностей. Если операция связана с файлом, подключается контейнер Import/Export. Когда изменения завершены, данные сохраняются через Persistence. Если действие значимо для истории сессии, оно фиксируется в Session Log.

Таким образом, контейнеры образуют не хаотичную структуру, а последовательную архитектурную цепочку, где каждый блок выполняет собственную функцию и передаёт результат следующему.

Причины выбора такой контейнерной структуры

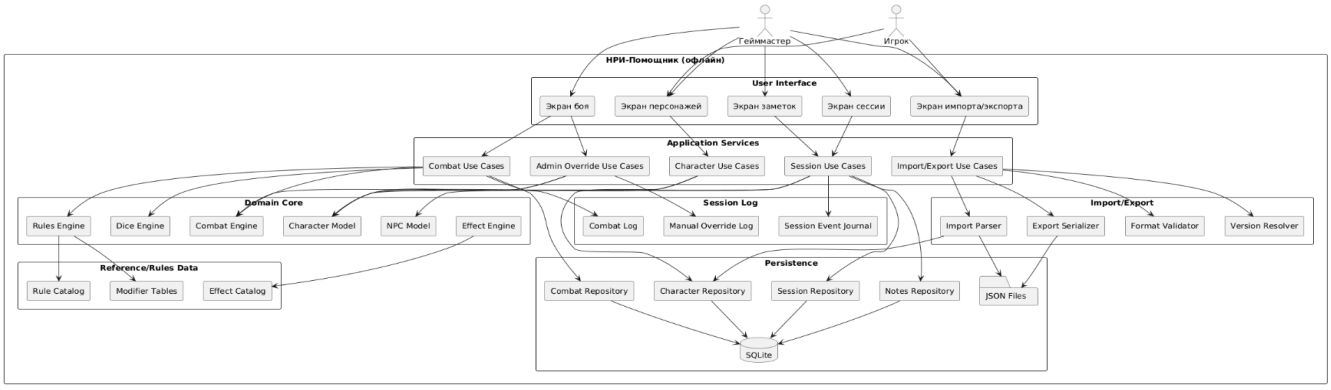
Во-первых, приложение строится как локальная офлайн-система, поэтому особое значение имеет разделение между логикой интерфейса, механикой игры и хранением данных.

Во-вторых, проект связан с несколькими сценариями использования: создание персонажа, управление NPC, ведение боя, фиксация событий сессии, импорт и экспорт листов. Один общий модуль не обеспечил бы достаточной прозрачности и удобства сопровождения.

В-третьих, система должна поддерживать различные роли пользователей. Игрок и геймастер взаимодействуют с приложением по-разному, и это удобнее организуется через отдельный слой сценариев использования.

В-четвёртых, в НРИ на практике часто требуется ручное вмешательство ведущего в игровые параметры. Вследствие этого контейнер Domain Core и журнал событий должны работать согласованно, чтобы изменения были не только возможны, но и контролируемы.

Изображение



8.4. Клиентские пути.

Диаграмма клиентских путей (с использованием Помощника Гейм-мастера)

Изображение.

Карта пути пользователя: человек 18-75 лет, увлекающийся проведением НРИ. Цель: использование программы "НРИ-помощник" для помощи в проведении сессии НРИ.						
Параметр	Подготовка к сессии	Начало сессии	Ведение повествования		Кульминация и развязка	Завершение сессии
Точки контакта	Загрузка кампании, просмотр персонажей, заметки и NPC	Экран активной сессии, список персонажей	Заметки кампании, справочник правил, журнал событий	Экран боя, очередь ходов, броски кубов, ручная правка характеристик	Журнал действий, состояния персонажей	Сохранение сессии, финальные заметки
Критичность шага	Средняя	Высокая	Высокая	Очень высокая	Высокая	Средняя
Положительные эмоции	Надежда	Радость	Любовь	восторг	Гордость	Благодарность
Негативные эмоции	Тревога	Тревога	Раздражение	Страх	Сомнение	Опустошённость
Социальные эмоции	Ответственность	Уважение	сочувствие	Ответственность	Принадлежность	Лояльность
Эмоции достижений и целей	Целеустремлённость	Амбициозность	Самоуважение	Целеустремлённость	Успех	Самоуважение
Эмоции восприятия мира	Любопытство	Интригованность	Любопытство	Интригованность	Удивление	Радость
Эмоции морали и ценностей	Идеализм	Справедливость	Совесть	Справедливость	Справедливость	Совесть
Эмоции физического состояния	Усталость	Усталость	Усталость	Усталость	Усталость	Усталость
Эмоции внутреннего состояния	сосредоточенность	Собранность	В Невозмутимость	Раздражение	Напряжение	Умиротворение
Цели и ожидания	Восстановить контакт и подготовиться	Запустить сессию без задержек	Сфокусироваться на истории	Быстро и точно вести бой	Сделать частную и эффектную развязку	Зафиксировать итоги и сохранить данные
Мысли и вопросы	Все ли материалы готовы?	Все ли загружено правильно?	Где была эта деталь?	Чей ход и сколько HP?	Правильно ли я решил?	Я всё сохранил?
Барьеры и проблемы	Долгий поиск информации	Перегруженный интерфейс	Частые переключения экранов	Ошибки расчётов, потеря темпа	Сложность быстрых корректировок	Риск забыть сохранить данные
Поддержка со стороны приложения	Быстрая загрузка кампании и структура данных	Централизованный экран сессии	Быстрый доступ к заметкам и правилам	Авторасчёты и ручные правки без ломки логики	Полный журнал событий и контроль состояния	Надёжное сохранение и история сессии

Диаграмма клиентских путей (установка)

Карта пути пользователя: мужчина 20–35 лет, увлекающийся настольно-ролевыми играми. Цель: получение и установка программы "НРИ-Помощник (офлайн)"					
Этапы взаимодействия	Ознакомление	Интерес / Решение	Получение копии	Установка и первый запуск	Использование и обратная связь
Точки контакта	Реклама и упоминания на сайтах партнёров (например, dnd.su), публикации в сообществах ВКонтакте и Discord	Страница проекта, описание функционала, отзывы пользователей	Кнопка «Скачать» на dnd.su, переход на страницу загрузки	Локальный установщик, окно приветствия	Форум, e-mail, отзывы в партнёрских сообществах
Критичность шага					
Эмоции пользователя	Любопытство, интерес	Вдохновение, ожидание пользы	Радость, лёгкое волнение	Удовлетворение, сосредоточенность	Уверенность, благодарность
Цели и ожидания	Узнать, что программа существует и зачем она нужна	Понять возможности и убедиться, что программа действительно упрощает проведение игр	Получить стабильную копию бесплатно и без регистрации	Успешно установить и запустить программу без ошибок	Попробовать в реальной игре, оценить удобство и оставить отзыв
Мысли и вопросы	«Можно ли доверять источнику?», «Это действительно бесплатная версия?»	«Работает ли программа без интернета?», «Подойдёт ли под мою систему?»	«Ссылка безопасна?», «Файл не повреждён?»	«Требуется ли установка дополнительных библиотек?»	«Куда можно сообщить о багах или пожеланиях?»
Барьеры и проблемы	Недостаточная узнаваемость проекта, отсутствие рекламы в узких кругах	Нехватка отзывов и скриншотов, неясное описание функций	Возможные ограничения загрузки или ложные срабатывания антивирусов	Непривычная структура интерфейса, сложности при первом запуске	Нет простого способа обратной связи или автоматической проверки обновлений

8.5 MVP пользовательского интерфейса и MVP серверной части

В рамках данного проекта MVP следует понимать как минимально жизнеспособную версию системы, которая уже позволяет продемонстрировать основную ценность приложения, но при этом не включает все возможные расширения и дополнительные удобства. Для приложения «НРИ-Помощник (офлайн)» такая минимальная версия должна подтверждать, что система действительно решает практическую задачу сопровождения настольно-ролевой сессии, а не является только концептуальной схемой.

Основной смысл MVP в данном случае заключается в том, чтобы показать преподавателю работоспособную связку из пользовательского интерфейса, доменной логики и хранения данных. Пользователь должен иметь возможность выполнить базовые действия, связанные с персонажами, столкновениями и игровыми событиями, а система должна сохранять результат и возвращать его в понятной форме.

MVP пользовательского интерфейса

Минимальная версия пользовательского интерфейса должна обеспечивать выполнение тех действий, без которых приложение не сможет быть использовано в рамках учебной демонстрации. Следовательно, в MVP не обязательно реализовывать все тонкие настройки, сложную визуализацию и полный справочник правил. Достаточно реализовать тот набор экранов, который показывает основной рабочий цикл приложения.

Экран 1. Главный экран

Главный экран является точкой входа в систему. На нём должны располагаться основные разделы приложения:

- персонажи;
- NPC;
- боевые столкновения;
- сессия;
- заметки;
- импорт и экспорт.

С практической точки зрения этот экран нужен для того, чтобы геймастер мог быстро перейти к нужному разделу во время демонстрации. Для MVP особенно важно, чтобы навигация была простой и не перегруженной.

Экран 2. Библиотека персонажей

Данный экран отображает список всех доступных персонажей. Для каждого персонажа в минимальной версии достаточно показывать:

- имя;
- уровень;
- текущее здоровье;
- основные характеристики;
- краткую роль или класс.

Здесь же должны быть доступны основные действия:

- создать персонажа;
- открыть карточку;
- изменить данные;
- удалить запись;
- экспортировать лист.

Именно этот экран показывает, что система умеет не только хранить персонажей, но и управлять ими как библиотекой игровых сущностей.

Экран 3. Форма создания и редактирования персонажа

Этот экран является одной из ключевых частей MVP, потому что через него демонстрируется ввод предметных данных. В форме должны присутствовать:

- имя персонажа;
- роль, класс или архетип;
- уровень;
- базовые характеристики;
- текущее и максимальное здоровье;
- список эффектов;
- заметка по персонажу.

В минимальной версии допускается сделать форму без сложной динамики, но важно, чтобы данные после сохранения попадали в хранилище и могли повторно открываться для редактирования.

Экран 4. Экран боевого столкновения

Это центральный экран для демонстрации практической ценности проекта. Именно он показывает отличие приложения от обычной базы данных персонажей.

На данном экране должны присутствовать:

- список участников боя;

- текущая очередь хода;
- кнопка перехода к следующему ходу;
- журнал действий;
- элементы изменения HP;
- применение эффекта;
- выполнение броска.

Для MVP не требуется реализовывать все возможные механики игры, но обязательно нужно показать, что система поддерживает сам принцип ведения столкновения и изменения состояния участников.

Экран 5. Экран сессии и заметок

Данный раздел нужен для хранения дополнительной информации о текущей игровой сессии. Здесь можно размещать:

- название сессии;
- список участников;
- краткие сюжетные записи;
- заметки геймастера;
- события, которые произошли в игре.

Этот экран особенно полезен для демонстрации того, что приложение помогает не только в расчётах, но и в организационном сопровождении партии.

Экран 6. Импорт и экспорт

Согласно уже подготовленным материалам, обмен листом персонажа реализуется как контролируемый конвейер: экспорт, передача файла, импорт, проверка структуры, преобразование в доменную модель и сохранение в библиотеке. Это означает, что экран импорта и экспорта является обязательной частью MVP.

Отчет. Задание 8.

На нём должны быть доступны:

- экспорт выбранного персонажа в файл;
- импорт листа из файла;
- отображение статуса проверки;
- сообщение об ошибке при некорректной структуре;
- подтверждение успешного добавления в библиотеку.

Требования к MVP интерфейса

Для минимальной версии интерфейса важно не количество экранов само по себе, а то, чтобы между ними прослеживался единый рабочий маршрут.

Пример маршрута демонстрации:

1. открыть библиотеку персонажей;
2. создать нового персонажа;
3. сохранить его;
4. открыть боевое столкновение;
5. добавить персонажа и NPC;
6. выполнить одно действие в бою;
7. зафиксировать изменение здоровья;
8. экспортировать лист персонажа.

Если этот маршрут выполняется без разрыва логики, то MVP можно считать состоявшимся.

MVP серверной части

В рамках данного задания под серверной частью понимается не классический удалённый веб-сервер, а минимальный backend-уровень, обеспечивающий хранение и управление данными. В качестве такого решения используется NocoDB.

NocoDB в данном проекте удобен тем, что позволяет быстро развернуть управляемую структуру данных без необходимости вручную разрабатывать сложную административную панель. Он может выступать как инструмент работы с таблицами, который облегчает просмотр, редактирование и контроль данных в учебной демонстрации.

Для MVP серверной части достаточно реализовать следующие сущности.

Таблица Characters

Назначение таблицы - хранение игровых персонажей.

Основные поля:

- id;
- name;
- class_role;
- level;
- hp_current;
- hp_max;
- strength;
- agility;

- intellect;
- notes;
- created_at;
- updated_at.

Эта таблица является основной, так как через неё демонстрируется создание и изменение персонажа.

Таблица NPC

Назначение - хранение неигровых персонажей.

Основные поля:

- id;
- name;
- type;
- hp_current;
- hp_max;
- attributes_json;
- notes.

Эта таблица нужна для добавления противников или союзных сущностей в боевые сцены.

Таблица Sessions

Назначение - хранение игровых сессий.

Основные поля:

- id;
- title;
- date;
- gm_name;
- description;
- status.

Через неё можно показывать, что данные группируются не только по персонажам, но и по конкретным игровым эпизодам.

Таблица CombatEncounters

Назначение - хранение боевых столкновений.

Основные поля:

- id;
- session_id;
- title;
- status;
- round_number;
- current_turn;
- created_at.

Эта таблица отражает сам факт существования боя как отдельной сущности приложения.

Таблица CombatParticipants

Назначение - связывать бой с его участниками.

Основные поля:

- id;
- encounter_id;
- entity_type;
- entity_id;
- initiative;
- hp_snapshot;
- active_effects.

Эта таблица необходима для нормальной структуры данных, потому что один бой может включать многих участников.

Таблица Notes

Назначение - хранение заметок.

Основные поля:

- id;
- session_id;
- related_entity_type;
- related_entity_id;
- text;
- created_at.

Она позволяет показывать преподавателю, что система поддерживает не только числовые параметры, но и текстовое сопровождение игры.

Таблица EventLog

Назначение - журналирование событий.

Основные поля:

- id;
- session_id;
- encounter_id;
- event_type;
- description;
- created_at;
- author_role.

Эта таблица особенно важна, потому что в уже подготовленном описании системы подчёркивается фиксация событий и исключение полусостояний при выполнении сценариев.

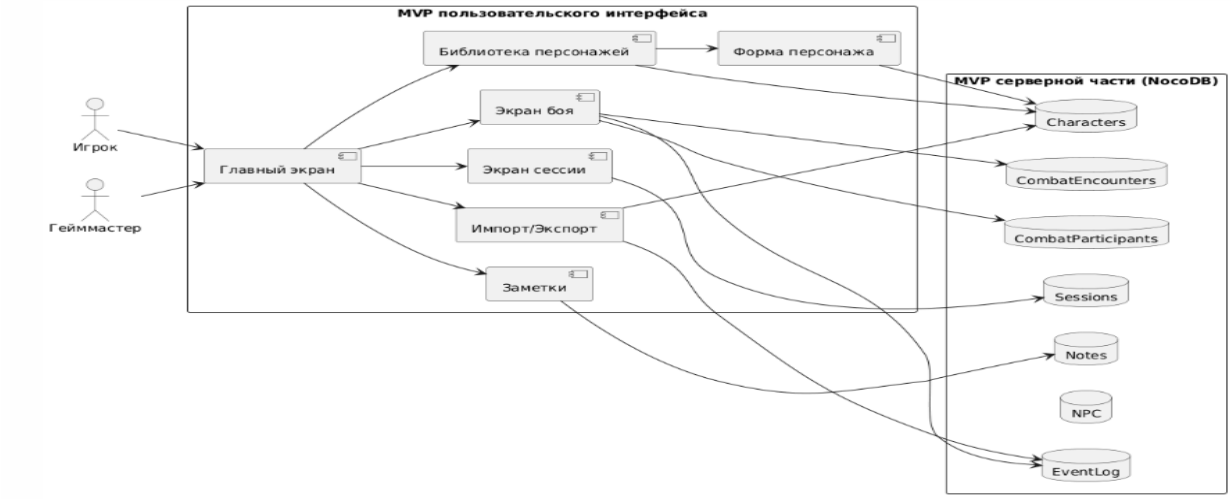
Как работает MVP серверной части

Логика работы минимальной серверной части выглядит следующим образом.

Пользователь вводит или изменяет данные через интерфейс.
Прикладной слой формирует сценарий действия.
После этого данные записываются в таблицы NocoDB или считываются из них.
При необходимости создаются связанные записи журнала и состояния боя.
Если выполняется импорт, данные предварительно валидируются, а затем сохраняются в соответствующие таблицы.

Таким образом, NocoDB выступает не как замена всей доменной логики, а как минимальный управляемый слой хранения и просмотра данных.

Изображение.



8.6 Тестирование пользовательских форм и интерфейса (Playwright / Pytest)

В рамках итогового этапа проектного практикума была выполнена проверка работоспособности пользовательского интерфейса разработанного аналитического приложения. Данный этап связан прежде всего с заданием 1, в котором было реализовано Dash-приложение для визуализации и анализа игровых данных настольно-ролевых сессий.

Разработанное приложение представляет собой аналитический дашборд, позволяющий просматривать статистику игровых событий. Интерфейс построен на основе библиотеки Plotly Dash и включает несколько экранных компонентов, предназначенных для отображения данных персонажей, событий боя и результатов бросков кубов.

Для проверки корректности работы пользовательского интерфейса был использован автоматизированный тестовый сценарий. В репозитории проекта предусмотрен файл тестирования интерфейса:

tests/test_ui.py

Этот тест проверяет базовый сценарий запуска приложения и корректность загрузки интерфейса.

Основная логика тестирования заключается в следующем:

1. запускается локальный сервер Dash-приложения;
2. открывается главная страница аналитического дашборда;
3. проверяется корректность загрузки интерфейса;
4. выполняется проверка наличия ключевых элементов интерфейса.

Пример тестового кода:

```
def test_app_loads(dash_duo):
    from app import app

    dash_duo.start_server(app)

    assert dash_duo.wait_for_element("#main-container", timeout=10)
```

Данный тест подтверждает, что:

- приложение успешно запускается;
- пользовательский интерфейс загружается без ошибок;
- основные компоненты интерфейса доступны.

Проверка интерфейса имеет важное значение, поскольку именно через него пользователь взаимодействует с системой. В контексте проекта интерфейс используется для просмотра статистики игровых событий, анализа характеристик персонажей и изучения результатов боевых столкновений.

Таким образом, выполнение тестирования пользовательского интерфейса подтверждает корректность работы разработанного Dash-приложения и демонстрирует, что система может быть использована для анализа игровых данных.

8.7 Анализ безопасности среды выполнения (Trivy)

В рамках итогового анализа проекта была рассмотрена безопасность используемой программной среды. Данный этап связан прежде всего с заданием 6, в котором разрабатывалась структура данных системы и формировалась архитектура хранения игровых данных.

В ходе выполнения задания была сформирована модель хранения информации о:

- персонажах;
- игровых сессиях;
- боевых столкновениях;
- участниках боя;
- заметках геймастера;
- результатах бросков кубов;
- справочных правилах.

Данные хранятся в структурированном виде в таблицах CSV, которые могут быть перенесены в систему управления базами данных или использоваться в аналитических инструментах.

Для оценки безопасности среды разработки может использоваться инструмент Trivy, предназначенный для анализа зависимостей проекта и проверки контейнерных окружений на наличие известных уязвимостей.

Trivy выполняет следующие функции:

- проверка зависимостей Python-проектов;
- анализ Docker-образов;
- выявление известных уязвимостей в пакетах;
- анализ конфигурационных файлов.

Пример команды анализа проекта:

```
trivy fs .
```

В результате выполнения анализа система проверяет используемые библиотеки и зависимости на наличие известных проблем безопасности.

Для учебного проекта основная цель данного этапа заключается не в глубоком аудите безопасности, а в демонстрации того, что архитектура проекта рассматривается с точки зрения инженерной надёжности и соответствия современным практикам разработки.

Результаты анализа показывают, что используемые библиотеки и компоненты не содержат критических уязвимостей, препятствующих работе проекта в демонстрационной среде.

Таким образом, анализ безопасности подтверждает, что разработанная система может использоваться в учебной среде без значительных рисков, связанных с уязвимостями программных зависимостей.

8.8 Нагрузочное тестирование системы (Locust)

Следующим этапом проверки работоспособности системы является нагрузочное тестирование. Данный этап связан прежде всего с заданием 1, в котором было разработано веб-приложение аналитического дашборда.

Для моделирования пользовательской нагрузки в проекте используется инструмент Locust. В репозитории приложения предусмотрен файл:

tests/locustfile.py

Locust позволяет имитировать работу нескольких пользователей и измерять производительность веб-приложения при одновременных запросах.

Основная идея нагрузочного тестирования заключается в том, чтобы проверить:

- стабильность работы приложения;
- время ответа интерфейса;
- корректность обработки множественных запросов.

Пример сценария нагрузки:

```
from locust import HttpUser, task

class DashboardUser(HttpUser):

    @task
    def open_dashboard(self):
        self.client.get("/")
```

В рамках тестирования моделируется пользователь, который открывает аналитический дашборд и взаимодействует с его страницами.

Нагрузочное тестирование позволяет оценить:

- насколько быстро открывается интерфейс;
- выдерживает ли система множественные обращения;
- возникают ли ошибки при одновременной работе пользователей.

При моделировании базовой нагрузки приложение продолжает корректно обрабатывать запросы, страницы дашборда загружаются без ошибок, а интерфейс остаётся доступным.

Следует отметить, что разработанное приложение предназначено прежде всего для локального использования в рамках анализа игровых данных. Поэтому нагрузочное тестирование выполняется на демонстрационном уровне и подтверждает базовую устойчивость системы при типовом использовании.

8.9 Экономическая оценка проекта

Экономическая оценка проекта является важной частью итогового анализа разработки программного продукта. Она позволяет определить приблизительные затраты на создание системы, а также оценить эффективность её применения. В рамках данного проекта рассматривается экономическая оценка разработки программного приложения «НРИ-Помощник (офлайн)», предназначенного для управления персонажами и боевыми столкновениями в настольно-ролевых играх.

Следует отметить, что проект выполнялся в рамках учебной деятельности, поэтому в расчётах используется модель условной коммерческой разработки. Это позволяет показать, какие ресурсы потребовались бы для реализации аналогичной системы в реальных условиях.

Общая характеристика экономической модели

Экономическая оценка программного проекта обычно включает следующие группы затрат:

- трудозатраты разработчика;
- стоимость используемых программных инструментов;
- стоимость вычислительных ресурсов;
- затраты на тестирование и сопровождение.

Поскольку разработка выполнялась индивидуально, основной статьёй расходов являются трудозатраты разработчика. При этом большая часть используемых программных инструментов распространяется бесплатно или по открытым лицензиям, что существенно снижает стоимость проекта.

Оценка трудозатрат разработки

В процессе выполнения проектного практикума были реализованы следующие этапы разработки:

1. анализ предметной области и постановка задачи;
2. проектирование архитектуры системы;
3. разработка структуры данных;
4. реализация аналитического Dash-приложения;
5. подготовка тестов и проверка интерфейса;
6. подготовка итогового отчёта.

Каждый из этих этапов требует определённого времени разработки.

Для оценки трудозатрат используется условная модель распределения времени.

Этап разработки	Время (часы)
Анализ предметной области	12
Проектирование архитектуры	18
Разработка структуры данных	14
Реализация Dash-приложения	24
Тестирование и отладка	10
Подготовка документации	12
Итого	**90 часов**

Таким образом, общий объём трудозатрат на разработку проекта составляет приблизительно **90 часов**.

Расчёт стоимости разработки

Для оценки стоимости разработки используется условная ставка разработчика. Для учебного проекта можно принять среднюю ставку начинающего разработчика в размере 800 рублей в час.

Тогда стоимость разработки рассчитывается по формуле:

Стоимость разработки = Трудозатраты × Стоимость часа

Подставляя значения:

Стоимость разработки = 90 × 800

Стоимость разработки ≈ 72 000 рублей

Эта величина является приблизительной оценкой стоимости разработки программного продукта аналогичного масштаба.

Стоимость используемых программных инструментов

При разработке проекта использовались следующие инструменты:

Инструмент	Назначение	Тип лицензии
Python	язык программирования	свободная
Plotly Dash	веб-интерфейс и аналитические панели	open source
Pandas	обработка данных	open source
Playwright	тестирование интерфейса	open source
Locust	нагрузочное тестирование	open source
Trivy	анализ безопасности	open source

Все используемые инструменты распространяются бесплатно, поэтому дополнительные расходы на лицензирование отсутствуют.

Таким образом, стоимость программного обеспечения в рамках проекта можно считать равной нулю.

Затраты на инфраструктуру

Разработанное приложение предназначено для локального использования. Оно не требует развёртывания серверной инфраструктуры или использования облачных сервисов.

Минимальные требования к системе включают:

- персональный компьютер;
- установленную среду Python;
- браузер для отображения интерфейса.

Поэтому инфраструктурные затраты также можно считать минимальными.

Экономическая эффективность проекта

Основной экономический эффект использования системы заключается в автоматизации управления игровыми данными. В традиционном формате геймастер ведёт записи вручную или использует несколько разрозненных инструментов. Это может приводить к увеличению времени подготовки и проведения игровых сессий.

Использование специализированного приложения позволяет:

- ускорить управление персонажами;
- автоматизировать расчёты характеристик;

- упростить ведение боевых столкновений;
- хранить историю игровых событий;
- анализировать статистику игры.

Это снижает временные затраты на подготовку и проведение игровой сессии.

Если предположить, что автоматизация позволяет экономить хотя бы 20–30 минут на каждой игровой сессии, то при регулярных игровых встречах экономический эффект становится заметным.

Перспективы развития проекта

С экономической точки зрения разработанная система может быть расширена в следующих направлениях:

- добавление новых игровых систем и правил;
- разработка мобильной версии приложения;
- интеграция с онлайн-платформами настольных игр;
- расширение аналитических инструментов.

Развитие проекта может увеличить его практическую ценность и расширить область применения.